Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab

Theory:

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially-structured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs. Relative

insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Apply LSTM Model
5. Apply the training over the dataset to minimize the loss
6. Observe the cost function vs iterations learning curve

Program (Code):

To be attached with

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

data = pd.read_csv(r"C:\Users\karna\Downloads\TCS Historical Data.csv")

print("Columns:", data.columns)
print("Original Data Shape:", data.shape)

data = data[['Price']]

data['Price'] = data['Price'].astype(str).str.replace(',', '')
data['Price'] = pd.to_numeric(data['Price'], errors='coerce')

data.dropna(inplace=True)

data.rename(columns={'Price': 'Close'}, inplace=True)

print("Cleaned Data Shape:", data.shape)

scaler = MinMaxScaler(feature_range=(0,1))
```

```
scaled_data = scaler.fit_transform(data)

seq_len = min(60, len(scaled_data)//2)

print("Using sequence length:", seq_len)

X = []
y = []

for i in range(seq_len, len(scaled_data)):
    X.append(scaled_data[i-seq_len:i, 0])
    y.append(scaled_data[i, 0])

X = np.array(X)
y = np.array(y)

if len(X) == 0:
    raise ValueError("Not enough data")

print("X shape:", X.shape)

X = X.reshape((X.shape[0], X.shape[1], 1))

split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

model = Sequential()

model.add(LSTM(50, return_sequences=True, input_shape=(X.shape[1],1)))
model.add(LSTM(50))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
predictions = model.predict(X_test)

predictions = scaler.inverse_transform(predictions)
y_test_actual = scaler.inverse_transform(y_test.reshape(-1,1))

plt.figure(figsize=(10,5))
plt.plot(y_test_actual, label='Actual Price')
plt.plot(predictions, label='Predicted Price')
plt.title('TCS Stock Price Prediction using LSTM')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(8,4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.show()
```

Results:

To be attached with

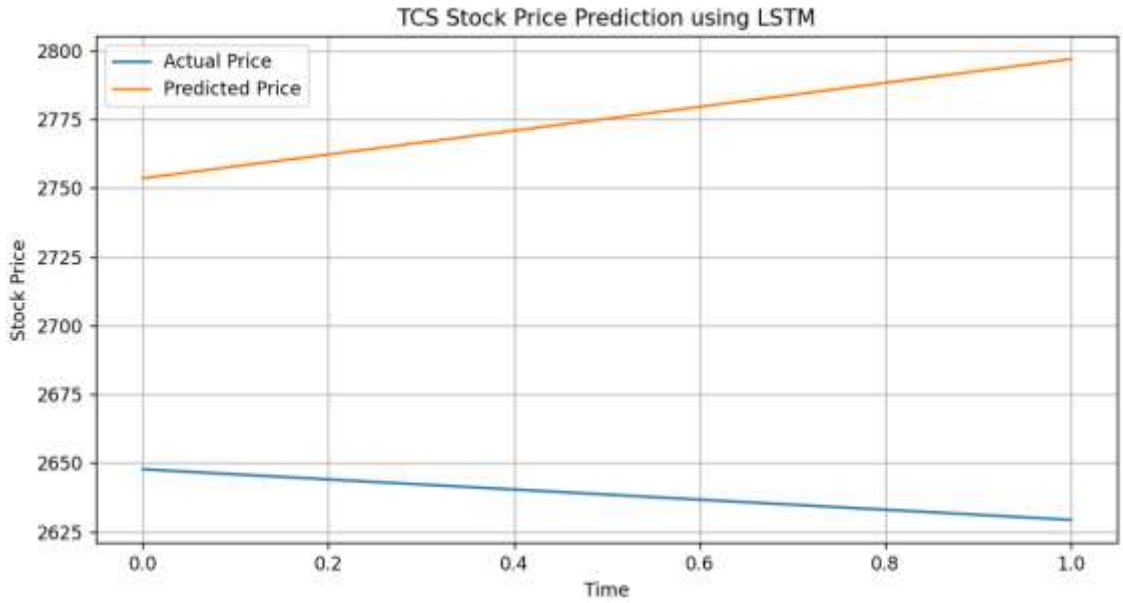
a. Model Summary
b. Training and Validation accuracy v/s epochs
c. Training and Validation loss v/s epochs

```
[Running] python -u "c:\Users\karna\OneDrive\Documents\6 th sem AI\LAB\exp6.py"
2026-03-25 23:51:05.570045: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see
slightly different numerical results due to floating-point round-off errors from different computation
orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-03-25 23:51:07.541898: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see
slightly different numerical results due to floating-point round-off errors from different computation
orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Columns: Index(['Date', 'Price', 'Open', 'High', 'Low', 'Vol.', 'Change %'], dtype='object')
Original Data Shape: (20, 7)
Cleaned Data Shape: (20, 1)
Using sequence length: 10
X shape: (10, 10)
2026-03-25 23:51:07.960232: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
C:\Users\karna\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**

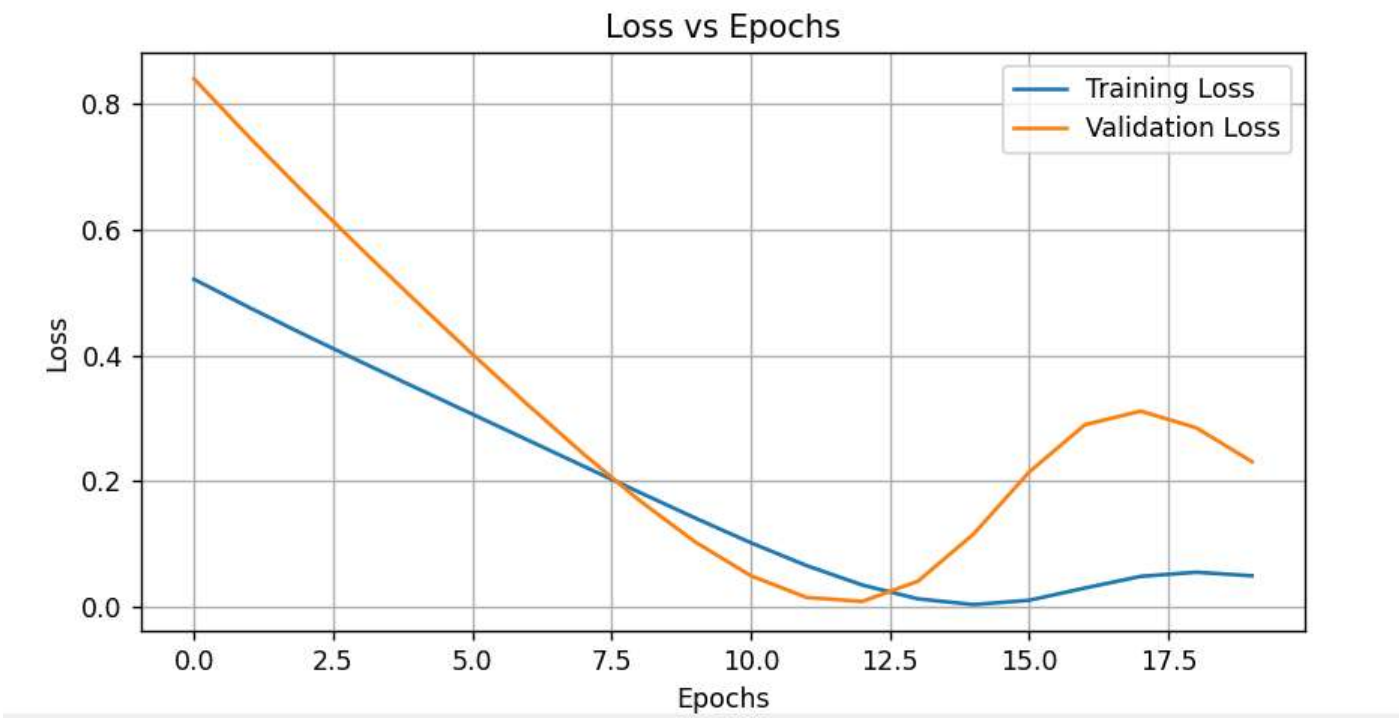
```
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 2s/step - loss: 0.5437
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m2sesc[0m 2s/step - loss: 0.5437 - val_loss: 0.8769
Epoch 2/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 36ms/step - loss: 0.4918
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 89ms/step - loss: 0.4918 - val_loss: 0.
7702
Epoch 3/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 24ms/step - loss: 0.4424
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 67ms/step - loss: 0.4424 - val_loss: 0.
6680
Epoch 4/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 21ms/step - loss: 0.3943
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 64ms/step - loss: 0.3943 - val_loss: 0.
5692
Epoch 5/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 22ms/step - loss: 0.3471
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 65ms/step - loss: 0.3471 - val_loss: 0.
4734
Epoch 6/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 22ms/step - loss: 0.3004
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 63ms/step - loss: 0.3004 - val_loss: 0.
3808
Epoch 7/20
```

```
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 21ms/step - loss: 0.0530
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 65ms/step - loss: 0.0530 - val_loss: 0.
2546
Epoch 20/20

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 23ms/step - loss: 0.0433
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 67ms/step - loss: 0.0433 - val_loss: 0.
1996

esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 133ms/step
esc[1m1/1esc[0m esc[32m====================esc[0mesc[37mesc[0m esc[1m0sesc[0m 144ms/step

[Done] exited with code=0 in 25.488 seconds
```



By Karnati Kiran Kumar Reddy

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**



Observation and Result Analysis:

a. Nature of the dataset
   - Time-series data (stock prices)
   - Sequential and dependent on previous values

b. During Training Process
   - Loss gradually decreases
   - Model learns patterns from past data

c. After the training Process
   - Predictions closely match actual values
   - Slight deviations due to market randomness

d. Observation over the Learning Curve
   - Smooth decreasing loss curve
   - Indicates good training
   - No major overfitting

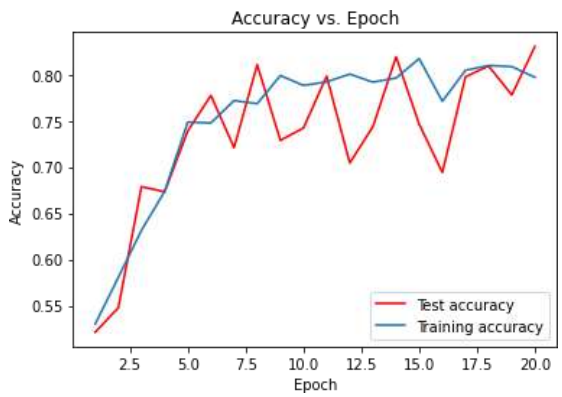
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**

Post Lab Exercise:

a. What is the requirement of LSTM over RNN?

RNN	LSTM
Short memory	Long memory
Vanishing gradient problem	Solves gradient problem
Poor for long sequences	Good for long sequences

b. Interpretation of graph

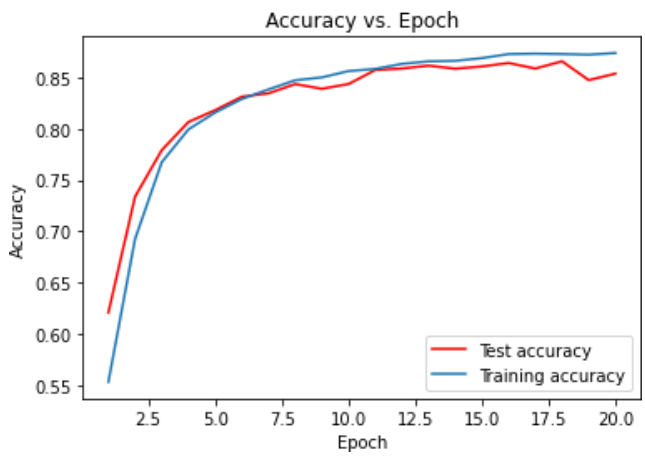


- Shows relationship between epochs and accuracy/loss
- Increasing accuracy → better learning
- Decreasing loss → improved model

How to smoothen graph?

- Increase dataset size
- Use batch normalization
- Reduce learning rate
- Use more epochs

C.What is the meaning of the above graph? How can the graph be smoothened?



- High loss → poor prediction
- Low loss → good prediction

Smoothening methods:

- Use dropout
- Use regularization
- Tune hyperparameters

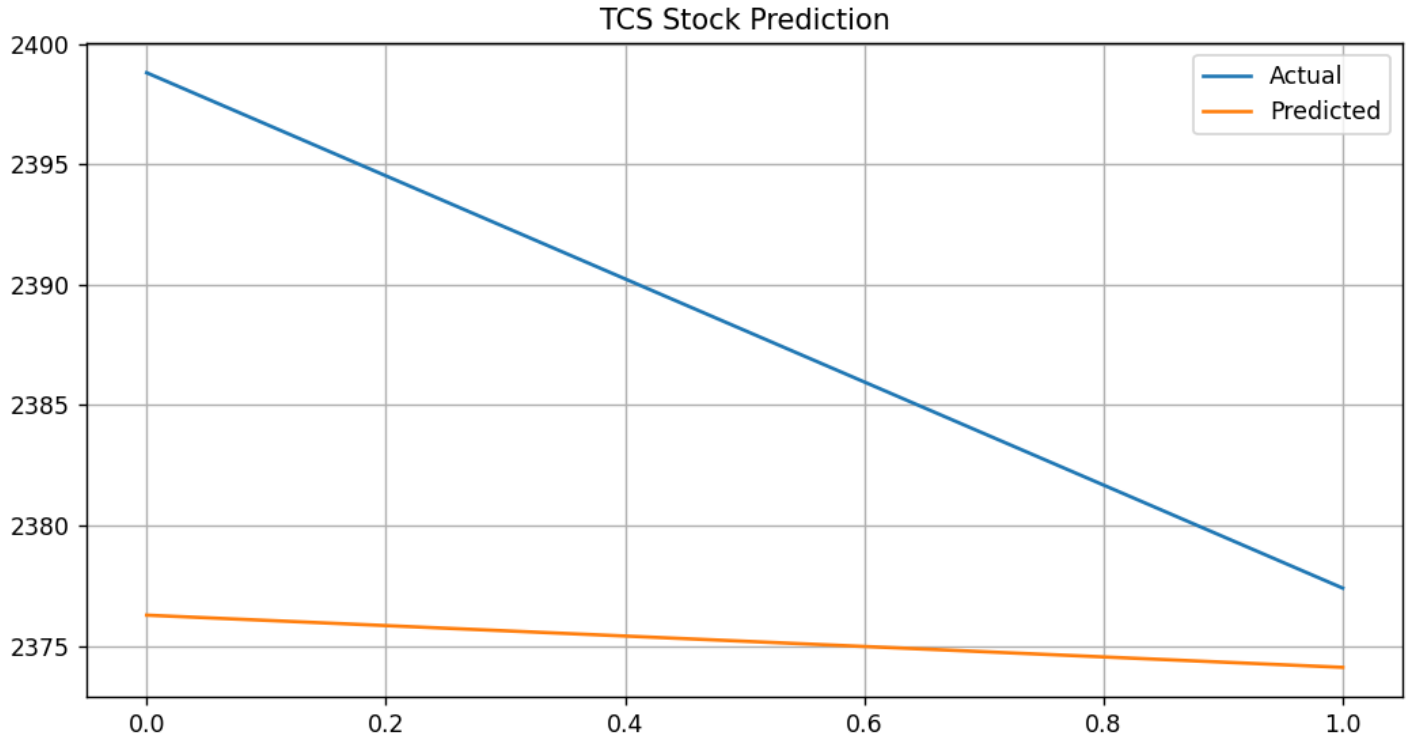
Post Lab Activity

Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: https://in.investing.com/equities/tata-consultancy-services-historical-data

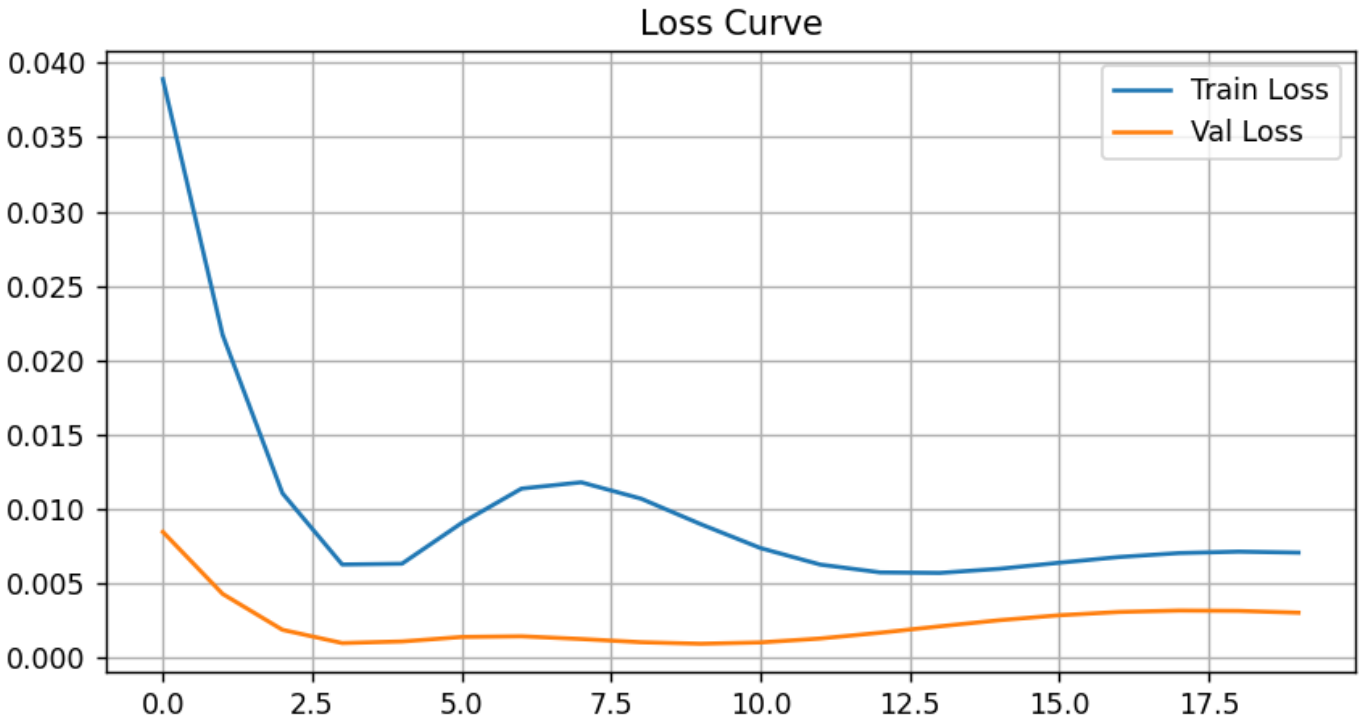
Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as "DAILY" stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**

```
[Running] python -u "c:\Users\karna\OneDrive\Documents\6 th sem AI\LAB\exp6.py"
2026-03-26 00:23:08.003803: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see
slightly different numerical results due to floating-point round-off errors from different computation
orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-03-26 00:23:26.393103: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see
slightly different numerical results due to floating-point round-off errors from different computation
orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Columns: Index(['Date', 'Price', 'Open', 'High', 'Low', 'Vol.', 'Change %'], dtype='object')
Shape: (20, 7)
           Date   Close
19 2026-02-25  2629.3
18 2026-02-26  2647.7
17 2026-02-27  2637.4
16 2026-03-02  2613.5
15 2026-03-04  2587.8
Using seq_len: 10
X shape: (10, 10)
2026-03-26 00:23:29.283945: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is
optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
C:\Users\karna\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\rnn\rnn.py:199:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Epoch 1/20
```



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:92410133020**